CH-10 Part - 3
In-Depth Study Notes: Advanced Table Constraints and CTAS (Create Table As Select)

As an expert Oracle SQL Developer, moving beyond basic table definitions requires understanding how to enforce complex, multi-column business rules and how to rapidly clone or materialize data sets for reporting. Based on the execution logs provided in the source file (CH10-4.TXT), these notes break down the architecture of **Table-Level Constraints**, **Composite Referential Integrity**, and the **CTAS (Create Table As Select)** methodology.

Concept 1: Table-Level & Composite Constraints

Core Purpose: While column-level constraints apply to a single attribute, business logic frequently demands rules that span multiple columns (e.g., "A specific combination of Standard, Division, and Roll Number must be unique"). **Table-Level Constraints** are defined at the end of the CREATE TABLE statement and can encompass multiple columns to create **Composite Keys** or complex CHECK conditions 1-3.

Keyword Breakdown:

- **CONSTRAINT**: Used to explicitly assign a custom name to the rule (e.g., SCOTT_SCHOOL_STD_DIV_RNO_CPK), making error tracking significantly easier 4.
- **PRIMARY KEY (col1, col2)**: Creates a Composite Primary Key. The individual columns can contain duplicate values across the table, but the *combination* of the columns must be strictly unique 2.
- **CHECK**: Evaluates boolean conditions. At the table level, it can cross-reference multiple columns in a single massive logical statement 3.
- **NOT NULL Constraint Limitation**: **NOT NULL cannot be defined at the table level** 1. Attempting NOT NULL(STD, DIV) at the end of a script will crash with an **ORA-00904: invalid identifier** error 5. It must strictly remain at the column level 6.

Comparison of Constraint Definitions:

Feature,Column-Level Constraint,Table-Level Constraint
Scope,Validates only the column it is attached to.,"Can validate one, several, or all columns."
Syntax Location,Immediately follows the Data Type.,Appended at the bottom of the column list.
NOT NULL Support,Allowed.,"Strictly Forbidden 1, 5."
Composite Keys,Impossible.,Mandatory for composite PK/UK/FK.

Syntax Breakdown:

```
CREATE TABLE table_name (
    col_1 data_type NOT NULL,
    col_2 data_type,
    col_3 data_type,
    CONSTRAINT custom_name PRIMARY KEY (col_1, col_2),
    CONSTRAINT custom_check CHECK (complex_logic_spanning_columns)
);
```

Detailed Example:A school needs a SCHOOL table where no two students can share the same Standard (STD), Division (DIV), and Roll Number (RNO). Additionally, different standards have different maximum roll number limits (e.g., Stds 1-4 max at 75, Stds 9-12 max at 45) 3, 7.

```
CREATE TABLE SCHOOL(
    STD NUMBER(2),
    DIV CHAR CONSTRAINT SCOTT_SCHOOL_DIV_CK#1 CHECK (DIV IN
('A','B','C','D')),
    RNO NUMBER(2),
    NM VARCHAR(10) CONSTRAINT SCOTT_SCHOOL_NM_CK#1 CHECK
(NM=UPPER(NM)),

    -- Table Level Constraints Below
    CONSTRAINT SCOTT_SCHOOL_STD_DIV_RNO_CPK PRIMARY KEY
(STD,DIV,RNO),
    CONSTRAINT SCOTT_SCHOOL_STD_RNO_CCK#1
    CHECK ((STD BETWEEN 1 AND 4 AND RNO BETWEEN 1 AND 45)
       OR  (STD BETWEEN 5 AND 8 AND RNO BETWEEN 1 AND 30)
       OR  (STD BETWEEN 9 AND 12 AND RNO BETWEEN 1 AND 15))
);
```

*[Execution: Enforces a composite primary key and validates complex tier-based rules. Inserting a Roll Number of 61 for Standard 1 violates the constraint and throws **ORA-02290** 8.]*

Execution Logic: During an INSERT or UPDATE, the Oracle Engine validates all column-level constraints first (data types, single-column checks, NOT NULL). Once the row passes, it evaluates the table-level composite checks and index lookups (Composite PKs/UKs). If any table-level check evaluates to FALSE, the entire transaction rolls back.

Pro-Tips/Best Practices:

- **Strict Naming Conventions:** The database generates arbitrary names like SYS_C0070216 if you omit the CONSTRAINT keyword 9. Always use a structured naming convention like SCHEMA_TABLE_COLUMNS_TYPE (e.g., SCOTT_SCHOOL_STD_DIV_RNO_CPK) to make your USER_CONSTRAINTS metadata views human-readable 4.

Concept 2: Composite Referential Integrity (FOREIGN KEY)

Core Purpose: When a parent table utilizes a Composite Primary Key, any child table referencing it must use a **Composite Foreign Key**. This guarantees that the child record aligns perfectly with the multi-column unique identifier of the parent 10.

Keyword Breakdown:

- **FOREIGN KEY (cols)**: Explicitly declares which columns in the *current* (child) table are acting as the relational anchor.
- **REFERENCES parent_table(cols)**: Points the anchor to the exact columns in the parent table.

Syntax Breakdown:

```
CREATE TABLE child_table (
    col_a data_type,
```

```
    col_b data_type,
    col_c data_type,
    FOREIGN KEY (col_a, col_b) REFERENCES parent_table (parent_col_1,
parent_col_2)
);
```

Detailed Example: We must create a FEES table that links to the SCHOOL table. Because SCHOOL uses the composite key (STD, DIV, RNO), the FEES table must explicitly reference all three simultaneously 4, 11.

```
CREATE TABLE FEES (
    STANDARD NUMBER(2),
    DIV CHAR,
    RNO NUMBER(2),
    AMOUNT NUMBER(6),
    FOREIGN KEY (STANDARD, DIV, RNO) REFERENCES SCHOOL(STD, DIV, RNO)
);
```

Execution Logic: When a row is inserted into FEES, the database intercepts the values for STANDARD, DIV, and RNO. It performs a rapid lookup against the B-Tree index powering the Primary Key of the SCHOOL table. If that exact 3-value combination does not exist, it throws an **ORA-02291: integrity constraint violated - parent key not found** 12.

Pro-Tips/Best Practices:

- **Data Type & Order Strictness:** The columns mapped in the FOREIGN KEY and REFERENCES clauses must perfectly match in both **data type** and **positional order**. Writing FOREIGN KEY (DIV, STANDARD, RNO) REFERENCES SCHOOL(STD, DIV, RNO) will instantly crash with **ORA-02267: column type incompatible with referenced column type** because it tries to map a CHAR to a NUMBER 11.

Concept 3: Table Cloning and Materialization (CTAS)

Core Purpose: The CREATE TABLE AS SELECT (CTAS) statement is a powerhouse DDL command. It allows developers to execute a complex query (involving aggregations, joins, logical evaluations, or pagination) and instantly materialize the result set into a brand new, physical table on the disk 13, 14.

Keyword Breakdown:

- **CREATE TABLE ... AS**: Instructs the database that the column definitions and data will be derived dynamically from the following SELECT statement.
- **SELECT ...**: The source query. It can be as simple as SELECT * or a highly complex multi-table JOIN with GROUP BY 15, 16.

Comparison of Table Creation Methods:

Method,Requires Explicit Column Definitions?,Copies Data?,Copies Constraints & Indexes?
Standard CREATE TABLE,Yes,No (Starts empty),Yes (Explicitly defined)
CTAS,No (Inherits from SELECT),Yes,Only NOT NULL constraints. (Primary/Foreign keys are lost).

Syntax Breakdown:

```
CREATE TABLE new_table_name AS
SELECT column_1, calculation AS alias
```

```
FROM source_table
[WHERE | JOIN | GROUP BY | ORDER BY ...];
```

Detailed Example: The business needs a static reporting table containing a list of employees making over 10,000, combining their first and last names into a single column, and including their lowercased email 14.

```
CREATE TABLE E4 AS
SELECT FIRST_NAME || ' ' || LAST_NAME AS FULLNAME,
       LOWER(EMAIL) AS EMAIL
FROM EMPLOYEES
WHERE SALARY > 10000;
```

Execution: Creates the table E4 and permanently inserts the 7 rows that matched the query into it 14, 17.

Execution Logic:

1. **Parse & Project:** The database parses the SELECT statement to determine the projection list. It uses the data types and column names from the SELECT to structure the new table.
2. **DDL Execution:** It creates the physical segment on the disk.
3. **Direct-Path Insert:** It executes the query and heavily optimizes the data insertion into the new table.

Pro-Tips/Best Practices:

- **The ORA-00998 Alias Trap:** If your CTAS SELECT statement includes calculations, concatenations, or functions (e.g., LOWER(EMAIL)), **you must provide a column alias**. If you fail to name the expression, the database doesn't know what to name the physical column, resulting in an **ORA-00998: must name this expression with a column alias** crash 18.
- **Missing Indexes:** A CTAS copies the physical data, but it drops all performance features from the source table. The new table will have no Primary Key, no Foreign Keys, and no B-Tree indexes 13. You must manually write ALTER TABLE statements after a CTAS to restore referential integrity and performance indexing.